

HTML & CSS

THE COMPLETE FIRE-POWERED NOTES

Web Basics | HTML | CSS | Responsive Design | Project Architecture | Exercises

01 WEB BASICS | 02 HTML | 03 CSS | 04 RESPONSIVE | 05 ARCHITECTURE | 06 EXERCISES

AI ZERO TO HERO - MODULE 27 - Comprehensive Full-Stack AI Engineer Roadmap

Sheharyar Sarmad

linkedin.com/in/sheharyar-sarmad-9b7736289 | github.com/Sheharyar-Sarmad/ai-zero-to-hero

Interactive PDF: click the table of contents to jump, tick the exercise checkboxes, open the bookmarks panel.

TABLE OF CONTENTS

Every entry is a clickable link. Use the PDF bookmarks panel for quick navigation too.

WEB BASICS 5

- The Internet vs The Web 5
- What Happens When You Open a Website 5
- Key Web Vocabulary 6
- Anatomy of a URL 6
- HTTP: The Language of the Web 6
- Frontend, Backend and Database 7
- The Three Pillars of the Frontend 7
- How a Browser Renders Your Page 8
- Your Developer Toolkit 8
- Set Up Your First Project 9

HTML 10

- What is HTML? 10
- The HTML Boilerplate 11
- Text, Headings and Formatting 11
- Links 12
- Images and Figures 12
- Lists 13
- Tables 13
- Forms 13
- Semantic HTML5 Layout 15
- Audio, Video and iframes 16
- The head: Metadata and SEO 16
- Global Attributes: id, class, data-* 16
- Accessibility Essentials 16
- HTML Entities 17
- HTML Best Practices and Common Mistakes 17

CSS 18

- What is CSS? 18
- Selectors 18
- Specificity, Cascade and Inheritance 19
- Colours and Gradients 19
- Units: px, rem, %, vw, vh, fr 20
- Typography 20
- The Box Model 21

Backgrounds, Borders and Shadows . . . 22
Display and Visibility . . . 22
Positioning and z-index . . . 22
Flexbox: One-Dimensional Layout . . . 23
Grid: Two-Dimensional Layout . . . 24
Interactive CSS: Hover, Transitions, Transforms . . . 25
Keyframe Animations: The Fire Flicker . . . 26
CSS Variables (Custom Properties) . . . 26
Modern CSS Worth Knowing . . . 27
A Solid Reset / Base Stylesheet . . . 27
Debugging CSS with DevTools . . . 27
CSS Common Mistakes . . . 27

RESPONSIVE DESIGN . . . 28

What is Responsive Design? . . . 28
The 10-Step Responsive Workflow . . . 28
Step 1: The Viewport Meta Tag . . . 29
Mobile-First vs Desktop-First . . . 29
Media Queries and Breakpoints . . . 29
Fluid Typography and Spacing . . . 30
Responsive Images and Video . . . 30
Layout Patterns That Always Work . . . 31
Responsive Navigation . . . 32
Tables, Forms and Touch Targets . . . 32
A Complete Mobile-First Page . . . 33
Responsive Testing Checklist . . . 34
Responsive Mistakes to Avoid . . . 34

PROJECT ARCHITECTURE . . . 35

Project Folder Structure . . . 35
Page Plan: What Goes on Each Page . . . 36
The Shared HTML Skeleton . . . 36
CSS Architecture: One File, Clear Order . . . 37
Reusable Components . . . 37
Git Workflow for This Project . . . 38
Deploy It for Free . . . 38
Final Quality Checklist . . . 38

EXERCISES . . . 39

How to Use This Section . . . 39
Level 0: Web Basics (answer in your notebook) . . . 39

Level 1: HTML Exercises 39
Level 2: CSS Exercises 40
Level 3: Responsive Design Exercises 40
Mini Projects 40
Frontend Mentor Practice Path 40
Final Challenge: The Fire Portfolio 41
Self-Test Quiz 41
Quiz Answers 42
One-Page Cheat Sheet 42
What Comes Next: JavaScript 43

PART 01

WEB BASICS

How the internet and websites actually work

01

The Internet vs The Web

The **internet** is the global network of connected computers: cables, routers, satellites and Wi-Fi. The **web** (WWW) is one service that runs on top of it: a huge collection of pages and apps that you open with a **browser**. Email, online games and video calls also use the internet, but they are not "the web".

As a web developer your job is to create the files that a server hands to a browser, and to make those files look good, work on every device and respond to the user.

What Happens When You Open a Website

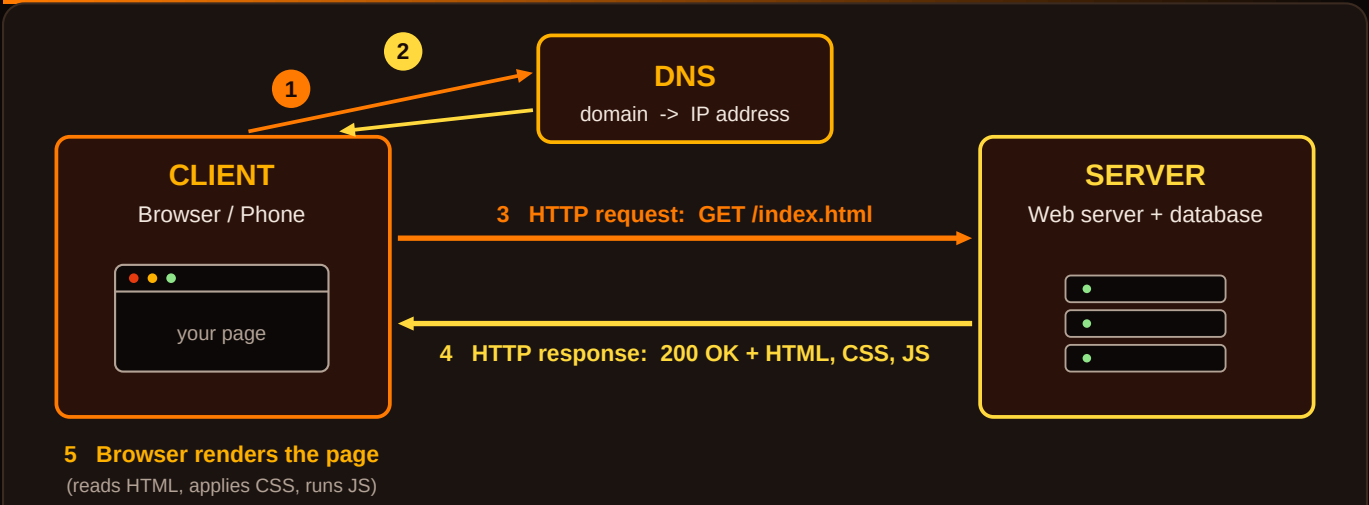


Figure 1 - The request / response cycle (every page load follows these 5 steps)

1. You type `example.com` in the address bar. The browser does not know where that is yet.
2. **DNS lookup:** the browser asks a DNS server, "which IP address belongs to this domain?"
3. The DNS server replies with an IP address such as `93.184.216.34`.
4. The browser sends an **HTTP request** to that IP: "please give me `/index.html`".
5. The server answers with an **HTTP response**: a status code plus the files. The browser then **renders** them as the page you see.

> **REMEMBER** A website is just a set of files (HTML, CSS, JavaScript, images, fonts) stored on a server and delivered to browsers on request. Nothing magic.

Key Web Vocabulary

Term	Meaning
Client	The device or browser that asks for things (your laptop, phone, Chrome, Brave).
Server	A computer that stores files / data and answers requests 24/7.
IP address	The numeric address of a machine on the network, e.g. `142.250.190.78`.
Domain name	Human-friendly name for an IP address, e.g. `github.com`.
DNS	The "phone book" that converts domain names into IP addresses.
HTTP / HTTPS	The protocol (rules) for client-server talk. HTTPS adds encryption (the padlock).
Hosting	Renting space on a server so your site is available to everyone.
Browser	Software that requests, receives and renders web content.
API	A doorway that lets one program request data from another (usually JSON).
Frontend / Backend	What users see and touch / the server logic and database behind it.

Anatomy of a URL



Figure 2 - A URL is a precise address for one resource on the web

- **Scheme** - the protocol: `http`, `https`, `ftp`, `mailto`.
- **Domain** - which server to talk to (`www.example.com`). `www` is a subdomain.
- **Port** - which door on the server. 443 for HTTPS and 80 for HTTP are the defaults and are hidden.
- **Path** - which resource on that server (`/blog/post.html`).
- **Query string** - extra data as `key=value` pairs, starting with `?` and joined with `&`.
- **Fragment** - jumps to an element with that `id` inside the page. Never sent to the server.

HTTP: The Language of the Web

Every request has a **method** (what you want to do) and every response has a **status code** (how it went).

Method	Purpose	Example
`GET`	Read / fetch data	Open a page, load a list of products
`POST`	Create new data	Submit a signup form
`PUT` / `PATCH`	Replace / partially update data	Edit your profile
`DELETE`	Remove data	Delete a comment

Code	Family	Meaning
`200 OK`	2xx Success	Everything worked, here is your content.
`201 Created`	2xx Success	A new resource was created (after POST).
`301 / 302`	3xx Redirect	The page moved permanently / temporarily.
`400 Bad Request`	4xx Client error	The request was malformed.
`401 / 403`	4xx Client error	Not logged in / not allowed.
`404 Not Found`	4xx Client error	That URL does not exist.
`500 Server Error`	5xx Server error	The server crashed while handling it.

A raw HTTP exchange

```
GET /index.html HTTP/1.1            <- request line (method, path, version)
Host: www.example.com               <- request headers
Accept: text/html

HTTP/1.1 200 OK                     <- response status line
Content-Type: text/html             <- response headers
Content-Length: 1256

<!DOCTYPE html>                     <- response body: the actual HTML
<html> ... </html>
```

🔥 Frontend, Backend and Database

Layer	Job	Typical tools
Frontend	Everything the user sees and clicks. Runs in the browser.	HTML, CSS, JavaScript, React, Next.js
Backend	Business logic, authentication, talks to the database. Runs on a server.	Node.js, Express, FastAPI, Python
Database	Stores data permanently.	PostgreSQL, MongoDB

This module is **100% frontend**: you will build static pages that look great on every screen. In the next modules JavaScript makes them interactive, and later you will connect them to a backend (MERN / PERN) and AI services.

🔥 The Three Pillars of the Frontend



Figure 3 - HTML builds it, CSS styles it, JavaScript brings it to life

REMEMBER Learn them in this order: **HTML first** (structure), **CSS second** (style), **JavaScript third** (behaviour). Each one builds on the one before.

How a Browser Renders Your Page

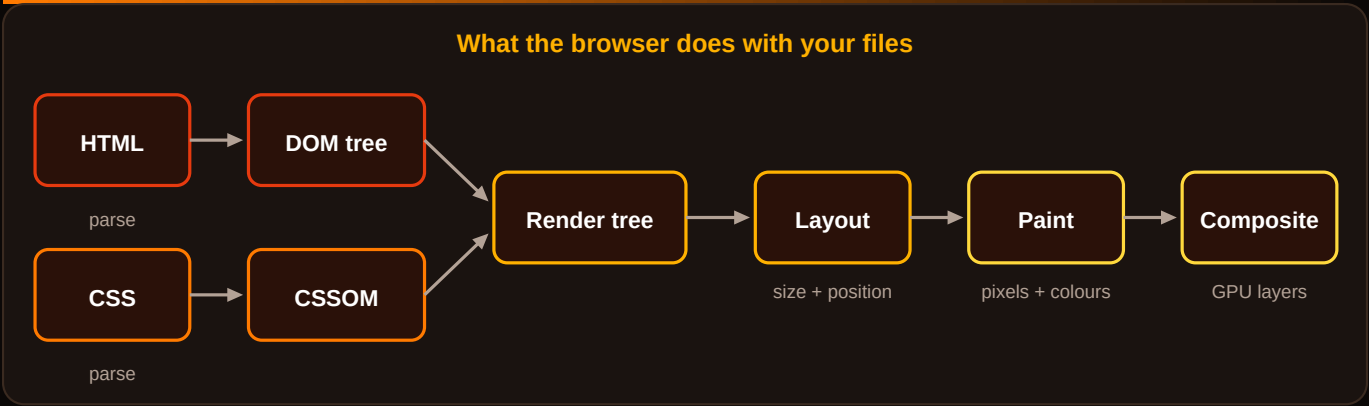


Figure 4 - The critical rendering path

- **DOM** (Document Object Model): the browser parses HTML into a tree of objects. JavaScript later edits this tree.
- **CSSOM**: the browser parses CSS into a second tree of style rules.
- **Render tree**: DOM + CSSOM combined. Only visible elements are included (`display: none` is skipped).
- **Layout**: calculates the exact size and position of every box.
- **Paint & Composite**: fills in pixels (colours, text, borders) and stacks layers on the GPU.

PRO TIP CSS in the `<head>` blocks rendering until it loads (so the page never flashes unstyled). That is why stylesheets go in the head and scripts go at the end of the body or use `defer`.

Your Developer Toolkit

Tool	What for
VS Code	Your code editor. Free, fast, huge extension ecosystem.
Live Server (extension)	Right-click `index.html` > Open with Live Server. The page reloads on every save.
Prettier (extension)	Auto-formats your code on save. Turn on "Format on Save".
Auto Rename Tag	Rename an opening tag and the closing tag changes too.
Emmet (built in)	Type `!` + Tab for the boilerplate; `ul>li*3` + Tab for a list.
Browser DevTools	Press `F12`. Inspect and edit HTML/CSS live. Your most important debugging tool.
Git + GitHub	Version control (Module 26). Commit after every small win.

DevTools tab	Use it to
Elements	See the DOM, edit HTML and CSS live, inspect the box model and computed styles.
Console	See errors and logs, run JavaScript.
Network	See every request: files, sizes, status codes, timing. Spot 404s.
Device toolbar (Ctrl+Shift+M)	Simulate phones and tablets to test responsive layouts.
Lighthouse	Audit performance, accessibility, SEO and best practices.

Set Up Your First Project

1. Create a folder with a short, lowercase name and **no spaces**: `portfolio`.
2. Open it in VS Code (`File > Open Folder`).
3. Create `index.html`. Browsers and servers treat `index.html` as the default page of a folder.
4. Create `style.css` next to it and link it from the HTML.
5. Right-click `index.html` > **Open with Live Server** and start building.

Terminal

```
mkdir portfolio
cd portfolio
code .                  # opens the folder in VS Code
git init                # start version control
```

WATCH OUT

File names are case-sensitive on most servers: `Style.css` and `style.css` are different files. Use lowercase, hyphens instead of spaces (`about-me.html`) and keep names descriptive.

PART 02

HTML

HyperText Markup Language: the structure of every web page

What is HTML?

HTML is a markup language, not a programming language. It does not calculate anything; it **labels content** so the browser knows what each piece is: a heading, a paragraph, a link, an image. Everything is built from **elements**.

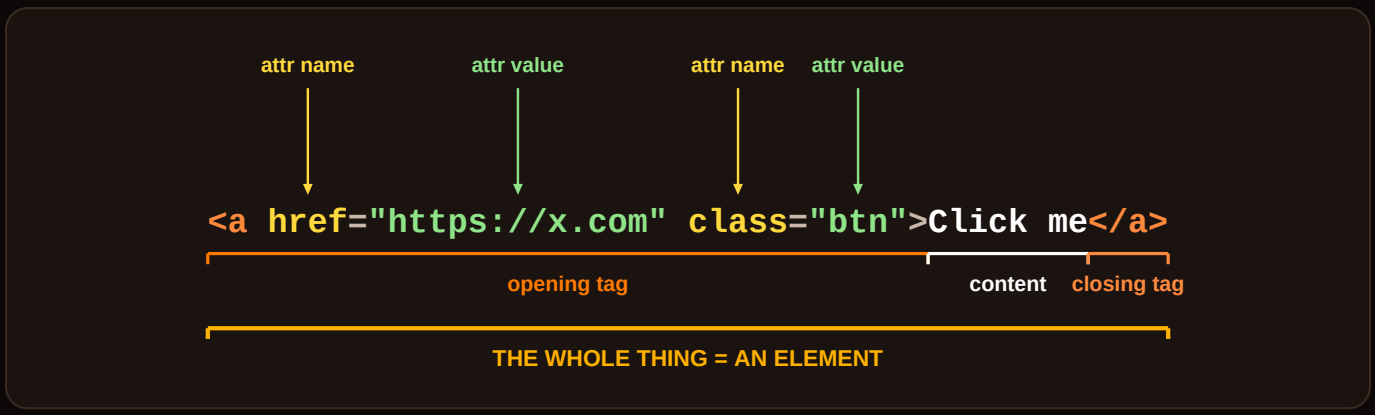


Figure 5 - Anatomy of an HTML element

- **Void elements** have no content and no closing tag: `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`.
- **Nesting:** elements live inside other elements. Close tags in the reverse order you opened them.
- HTML ignores extra spaces and line breaks. Ten spaces look the same as one.
- Tag names are case-insensitive, but always write them in **lowercase**.

HTML

```
<!-- Correct nesting -->
<p>This is <strong>very <em>important</em></strong> text.</p>

<!-- WRONG: tags cross each other -->
<p>This is <strong>very <em>important</strong></em> text.</p>
```

Display type	Behaviour	Examples
Block	Starts on a new line, takes the full width available.	`div` `p` `h1-h6` `ul` `li` `section` `form`
Inline	Flows inside a line of text, only as wide as its content.	`span` `a` `strong` `em` `img` `code`
Inline-block	Flows like inline but accepts width / height (set with CSS).	`button` `input` `select`

The HTML Boilerplate

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Portfolio</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Hello, world!</h1>

  <script src="js/app.js" defer></script>
</body>
</html>
```

Line	Purpose
`<!DOCTYPE html>`	Tells the browser this is modern HTML5 (not old quirks mode).
`<html lang="en">`	Root element. `lang` helps screen readers, translators and search engines.
`<head>`	Invisible info about the page: title, metadata, stylesheets.
`charset="UTF-8"`	Correct display of all characters, accents and emojis.
`name="viewport"`	**Essential for responsive design.** Makes phones use their real width.
`<title>`	Text on the browser tab and in Google results.
`<link rel="stylesheet">`	Connects your CSS file.
`<body>`	Everything visible on the page lives here.
`<script defer>`	Loads JavaScript without blocking the page.

PRO TIP In VS Code type `!` then press Tab. Emmet writes this whole boilerplate for you. Still, type it by hand a few times until you know every line.

Text, Headings and Formatting

HTML

```
<h1>Main title (only ONE per page)</h1>
<h2>Section heading</h2>
<h3>Sub-section heading</h3>

<p>A paragraph with <strong>bold</strong>, <em>italic</em> and
   <mark>highlighted</mark> text.<br>This starts a new line.</p>

<hr>
<blockquote>Talk is cheap. Show me the code.</blockquote>
<pre><code>function hi() {
  return "keeps spacing";
}</code></pre>
<p>H<sub>2</sub>O and E = mc<sup>2</sup>. <small>Fine print.</small></p>
```

Tag	Meaning	Notes
`<h1>-<h6>`	Headings, most to least important	Never skip levels. Choose by meaning, not by size.
`<p>`	Paragraph	Browser adds space before and after.

Tag	Meaning	Notes
`<strong>` / `<em>`	Strong importance / emphasis	Meaningful. Preferred over `<b>` / `<i>`.
` ` / `<hr>`	Line break / thematic divider	Do not use ` ` to create spacing; use CSS.
`<blockquote>`	Long quotation	Use `<q>` for short inline quotes.
`<pre>` + `<code>`	Preformatted code block	Keeps whitespace exactly as typed.

Links

HTML

```html
<a href="https://github.com/Sheharyar-Sarmad">External link</a>
<a href="about.html">Relative link (same folder)</a>
<a href="../index.html">Go up one folder</a>
<a href="/blog/post.html">Root-relative (from site root)</a>
<a href="#contact">Jump to the element with id="contact"</a>
<a href="mailto:me@example.com">Send email</a>
<a href="tel:+920000000000">Call me</a>
<a href="resume.pdf" download>Download file</a>
<a href="https://example.com" target="_blank" rel="noopener noreferrer">New tab</a>
```

WATCH OUT Always add `rel="noopener noreferrer"` to `target="_blank"` links. Link text must make sense alone: write "Read the guide", never "click here".

Images and Figures

HTML

```html
<img src="images/me.jpg" alt="Sheharyar smiling at his laptop"
     width="400" height="300" loading="lazy">

<figure>
  <img src="images/project1.png" alt="Screenshot of the weather app">
  <figcaption>Weather app, built with HTML and CSS.</figcaption>
</figure>

<picture>
  <source srcset="images/hero.webp" type="image/webp">
  <img src="images/hero.jpg" alt="Mountains at sunrise">
</picture>
```

Format	Best for	Notes
JPG	Photographs	Small files, no transparency.
PNG	Screenshots, graphics with transparency	Larger files, lossless.
SVG	Logos, icons, illustrations	Vector: razor sharp at any size, tiny file, stylable with CSS.
WebP / AVIF	Modern replacement for JPG / PNG	Much smaller files. Use `<picture>` with a fallback.

- `alt` **text is required.** Describe what the image shows. Use `alt=""` for purely decorative images.
- Set `width` and `height` so the browser reserves space and the page does not jump while loading.
- `loading="lazy"` delays off-screen images, which saves data and speeds up the page.

Lists

HTML

```
<ul>                        <!-- unordered: bullets -->
  <li>HTML</li>
  <li>CSS
    <ul><li>Flexbox</li><li>Grid</li></ul>   <!-- nested list -->
  </li>
</ul>

<ol start="1">              <!-- ordered: numbers -->
  <li>Learn</li><li>Build</li><li>Ship</li>
</ol>

<dl>                        <!-- description list -->
  <dt>HTML</dt><dd>Structure of the page</dd>
  <dt>CSS</dt><dd>Style of the page</dd>
</dl>
```

PRO TIP Navigation menus are semantically **lists of links**: a `<ul>` inside `<nav>`. You remove the bullets with CSS later.

Tables

HTML

```
<table>
  <caption>Weekly plan</caption>
  <thead>
    <tr><th scope="col">Day</th><th scope="col">Topic</th></tr>
  </thead>
  <tbody>
    <tr><td>Mon</td><td>HTML</td></tr>
    <tr><td>Tue</td><td>CSS</td></tr>
    <tr><td colspan="2">Wed: build the portfolio</td></tr>
  </tbody>
</table>
```

WATCH OUT Tables are for **data**, never for page layout. Layout is the job of CSS Flexbox and Grid.

Forms

HTML

```
<form action="/subscribe" method="post">
  <label for="name">Full name</label>
  <input type="text" id="name" name="name" placeholder="Ada Lovelace" required>

  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>

  <label for="level">Level</label>
  <select id="level" name="level">
    <option value="beginner" selected>Beginner</option>
    <option value="pro">Pro</option>
  </select>

  <fieldset>
    <legend>Newsletter</legend>
    <label><input type="radio" name="news" value="yes" checked> Yes</label>
    <label><input type="radio" name="news" value="no"> No</label>
  </fieldset>

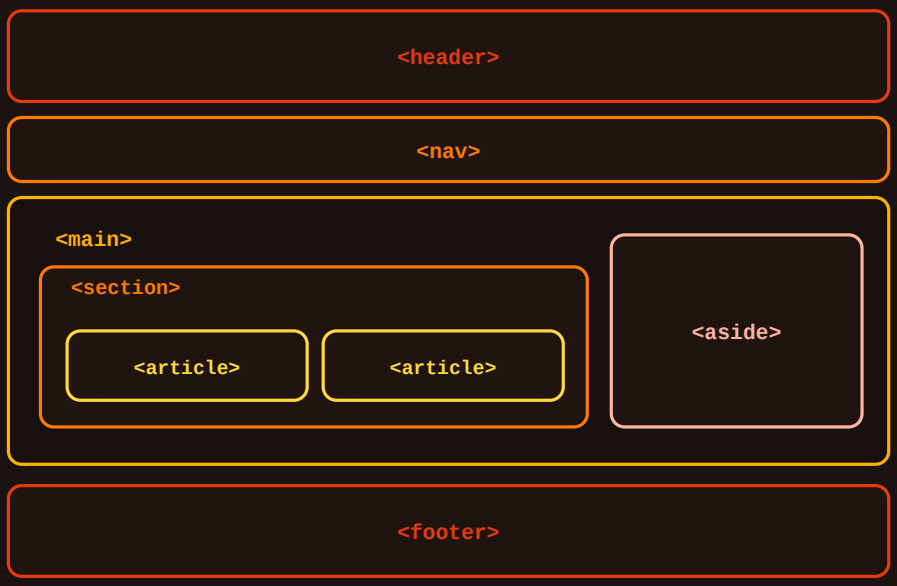
  <label><input type="checkbox" name="terms" required> I agree</label>
  <label for="msg">Message</label>
  <textarea id="msg" name="msg" rows="4"></textarea>
  <button type="submit">Send</button>
</form>
```

Input type	Gives you
`text` `password` `search`	Single-line text (password hides characters).
`email` `url` `tel` `number`	Built-in validation + the right mobile keyboard.
`date` `time` `color` `range`	Native pickers and sliders.
`checkbox` / `radio`	Many choices / exactly one choice (radios share the same `name`).
`file` / `hidden`	File upload / data the user does not see.

Attribute	Use
`name`	The key sent to the server. Without it, the field is not submitted.
`required` `minlength` `maxlength` `min` `max` `pattern`	Built-in validation before submitting.
`placeholder`	Hint text. It is **not** a replacement for a `<label>`.
`method` `action`	`GET` puts data in the URL (searches); `POST` sends it in the body (logins, signups).

REMEMBER Every input needs a `<label>` linked by `for` = the input's `id`. It makes the form usable with screen readers and gives you a bigger click target.

Semantic HTML5 Layout



Why semantic HTML?

Screen readers can jump between landmarks

Search engines understand the page better (SEO)

Code is easier to read and maintain

Avoids "div soup"

Figure 6 - Page landmarks: tags that describe their purpose

HTML

```
<body>
  <header>
    <h1>Sheharyar Sarmad</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="projects.html">Projects</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="about">
      <h2>About me</h2>
      <article><h3>My story</h3><p>...</p></article>
    </section>
    <aside>Related links</aside>
  </main>

  <footer><p>&copy; 2026 Sheharyar Sarmad</p></footer>
</body>
```

Tag	Use it for
`<header>` `<footer>`	Intro / closing content of the page or of a section.
`<nav>`	Major navigation links.
`<main>`	The unique main content. Only **one** per page.
`<section>`	A themed group of content, normally with a heading.
`<article>`	Self-contained content that could stand alone (blog post, project card).
`<aside>`	Side content related to the main content.
`<div>` / `<span>`	Generic containers with no meaning. Use only when no semantic tag fits.

Audio, Video and iframes

HTML

```
<video controls width="480" poster="images/cover.jpg">
  <source src="media/intro.mp4" type="video/mp4">
  Your browser does not support video.
</video>

<audio controls src="media/podcast.mp3"></audio>

<iframe src="https://www.youtube.com/embed/9cmy6AK3IBA"
        title="HTML and CSS one shot" width="560" height="315"
        allowfullscreen loading="lazy"></iframe>
```

Use `controls` so users can play and pause. Never autoplay with sound. Always give an `iframe` a `title` for accessibility.

The head: Metadata and SEO

HTML

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sheharyar Sarmad | AI & Full-Stack Engineer</title>
  <meta name="description" content="Portfolio of Sheharyar: AI, web and projects.">
  <meta name="theme-color" content="#ff7a00">
  <link rel="icon" href="images/favicon.png">
  <!-- Social media preview (Open Graph) -->
  <meta property="og:title" content="Sheharyar Sarmad - Portfolio">
  <meta property="og:image" content="https://example.com/preview.jpg">
  <link rel="stylesheet" href="css/style.css">
</head>
```

Global Attributes: id, class, data-*

Attribute	Purpose	Rule
`id`	Unique name for ONE element (link targets, JS)	Must be unique on the page.
`class`	Reusable label for styling many elements	Can be shared. Can hold several: `class="btn btn-big"`.
`style`	Inline CSS	Avoid. Keep CSS in its own file.
`title`	Tooltip on hover	Do not rely on it for important info.
`hidden`	Hides the element	Same as `display: none`.
`data-*`	Store custom data for JS	`data-price="19"` is read as `dataset.price`.
`tabindex`	Controls keyboard focus order	Use `0` or `-1` only. Avoid positive numbers.

Accessibility Essentials

- Use **semantic tags** so assistive technology can understand the page structure.
- Give every image meaningful `alt` text and every input a `<label>`.
- Keep a logical heading order: one `h1`, then `h2`, then `h3`, with no jumps.
- Make sure everything works with the **keyboard only** (Tab, Enter, Esc). Never remove focus outlines without replacing them.
- Colour contrast: text should be at least **4.5 : 1** against its background.

- Use `aria-label` only when there is no visible text, e.g. an icon-only button: `<button aria-label="Close">X</button>`.
- Declare the page language: `<html lang="en">`.

HTML Entities

Entity	Shows	Entity	Shows
`<`	<	`©`	(c) copyright
`>`	>	` `	non-breaking space
`&`	&	`"`	" quote
`♥`	heart symbol	`→`	right arrow

Use entities when you want to **display** reserved characters like `<` and `&` as text.

HTML Best Practices and Common Mistakes

Do this	Not this
Lowercase tags and attribute names, quoted values.	Mixed case, unquoted attributes.
Indent nested elements by 2 spaces.	One long unreadable line.
Semantic tags (`nav`, `main`, `article`).	Everything inside `div` ("div soup").
Real `<button>` for actions and `<a>` for navigation.	`<div onclick>` pretending to be a button.
One `h1`, logical heading order.	Headings chosen only for their size.
`alt` on every image, `label` on every input.	Missing alt, placeholder-only inputs.
CSS in `.css` files, linked in the head.	Inline `style` attributes and `<font>` tags.
Validate at validator.w3.org and fix errors.	Ignoring unclosed or mis-nested tags.

PART 03

CSS

Cascading Style Sheets: colour, layout, motion and fire

What is CSS?

CSS tells the browser how HTML elements should **look and be arranged**: colours, fonts, spacing, layout and animation. One stylesheet can restyle an entire website.

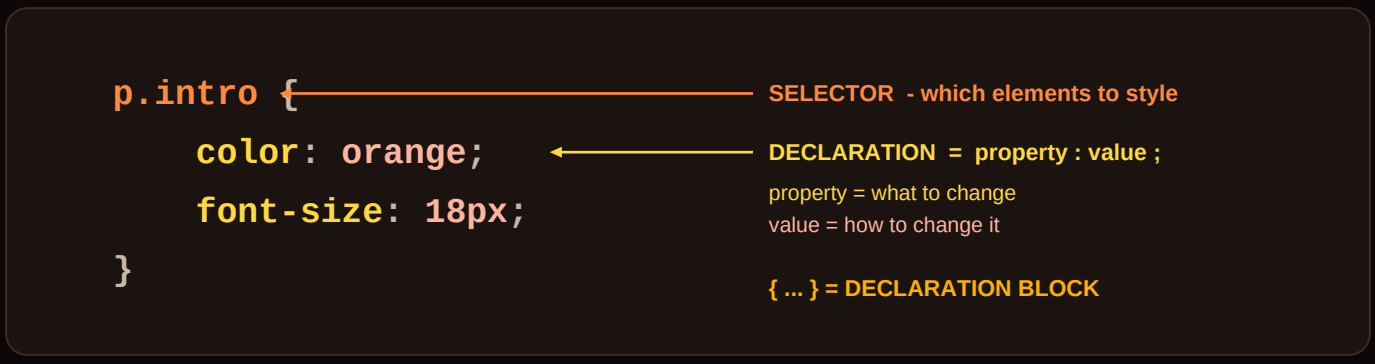


Figure 7 - Anatomy of a CSS rule

Method	Example	When to use
External (best)	`<link rel="stylesheet" href="css/style.css">`	Always. One file styles every page.
Internal	`<style> p { color: red; } </style>` in the head	Tiny demos, single-page experiments.
Inline	`<p style="color: red;">`	Avoid. Hard to maintain, overrides everything.

Selectors

CSS

```css
h1 { color: #ff7a00; }                /* element (type) selector   */
.card { padding: 16px; }              /* class selector            */
#hero { min-height: 100vh; }          /* id selector (use sparingly) */
* { box-sizing: border-box; }         /* universal selector        */
h1, h2, h3 { font-family: sans-serif; }   /* group: apply to all    */

nav a { text-decoration: none; }      /* descendant: any a in nav  */
ul > li { margin-bottom: 6px; }       /* child: direct children    */
h2 + p { font-size: 1.1rem; }         /* adjacent: p right after h2 */
h2 ~ p { color: gray; }               /* general sibling           */

a[target="_blank"] { color: orange; }  /* attribute selector       */
.btn.primary { background: crimson; }  /* element with BOTH classes */
```

Pseudo-class / element	Matches
`a:hover`	While the mouse is over the element.

Pseudo-class / element	Matches
`input:focus`	While the element has keyboard focus.
`button:active`	While it is being pressed.
`li:first-child` `li:last-child` `li:nth-child(odd)`	Position among siblings.
`input:checked` `input:disabled` `input:invalid`	Form states.
`p:not(.note)`	Everything except `.note`.
`p::first-line` `p::first-letter`	Part of an element (pseudo-element, double colon).
`.card::before` `.card::after`	Generated content you can style (needs `content: ""`).

🔥 Specificity, Cascade and Inheritance

When two rules target the same element, the browser picks a winner using three checks, in order:

1. **Importance:** `!important` beats normal declarations (avoid it).
2. **Specificity:** the more specific selector wins (table below).
3. **Source order:** if specificity is equal, the rule written **last** wins.

Selector type	Weight	Example
Inline style	1000	`style="color: red"`
ID	100	`#hero`
Class, attribute, pseudo-class	10	`.card` `[type="text"]` `:hover`
Element, pseudo-element	1	`p` `::before`

CSS

```css
p { color: blue; }            /* weight   1 */
.note { color: green; }       /* weight  10  -> beats p       */
p.note { color: orange; }     /* weight  11  -> beats .note   */
#intro { color: red; }        /* weight 100  -> beats all above */
```

REMEMBER **Inheritance:** some properties (`color`, `font-family`, `line-height`) pass from a parent to its children automatically. Others (`margin`, `border`, `background`) do not. Set fonts and colours once on `body` and let them flow down.

PRO TIP Keep specificity low and flat: style with single classes (`.card`, `.btn`). Avoid ids in CSS and avoid `!important`; both create "specificity wars".

🔥 Colours and Gradients

CSS

```css
.colors {
  color: tomato;                    /* named colour              */
  color: #ff7a00;                   /* hex (#rrggbb)             */
  color: rgb(255 122 0);            /* red green blue (0-255)    */
  color: rgb(255 122 0 / 50%);      /* with transparency         */
  color: hsl(28 100% 50%);          /* hue, saturation, lightness */
}

.fire-bg {
  background: linear-gradient(to top, #e63a0f, #ff7a00, #ffd93d);
}
.glow {
  background: radial-gradient(circle at 50% 100%, #ff7a00, #0b0807 70%);
}
```

HSL is the friendliest format for designers: change only the lightness to get lighter and darker versions of the same colour.

🔥 Units: px, rem, %, vw, vh, fr

Unit	Relative to	Use for
`px`	Screen pixel (fixed)	Borders, shadows, tiny details.
`rem`	Root (`html`) font-size, 16px by default	**Font sizes, spacing, media queries.** Respects user settings.
`em`	Font-size of the current element	Spacing that must scale with the element's text (buttons).
`%`	The parent element	Fluid widths: `width: 50%`.
`vw` / `vh`	1% of viewport width / height	Full-screen sections: `min-height: 100vh`.
`ch`	Width of the "0" character	Readable text width: `max-width: 65ch`.
`fr`	A share of free space in a grid	`grid-template-columns: 1fr 2fr`.

PRO TIP Rule of thumb: **rem** for type and spacing, **%** or **fr** for widths, **px** only for hairline details, **vh** for hero sections.

🔥 Typography

HTML

```html
/* in the HTML head: load a Google Font */
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="stylesheet"
  href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap">
```

CSS

```css
body {
  font-family: "Poppins", system-ui, sans-serif;   /* always add fallbacks */
  font-size: 1rem;              /* 16px base                        */
  line-height: 1.6;             /* unitless: 1.5 - 1.7 for body text */
  color: #f2e8df;
}
h1 {
  font-size: 2.5rem;
  font-weight: 700;             /* 400 normal, 700 bold             */
  letter-spacing: -0.02em;
  text-transform: uppercase;
  text-shadow: 0 0 18px #ff7a00;   /* glowing fire text             */
}
a { text-decoration: none; }
p { max-width: 65ch; text-align: left; }
```

The Box Model

Every element is a rectangular box made of four layers. Understanding this one idea solves most layout confusion.

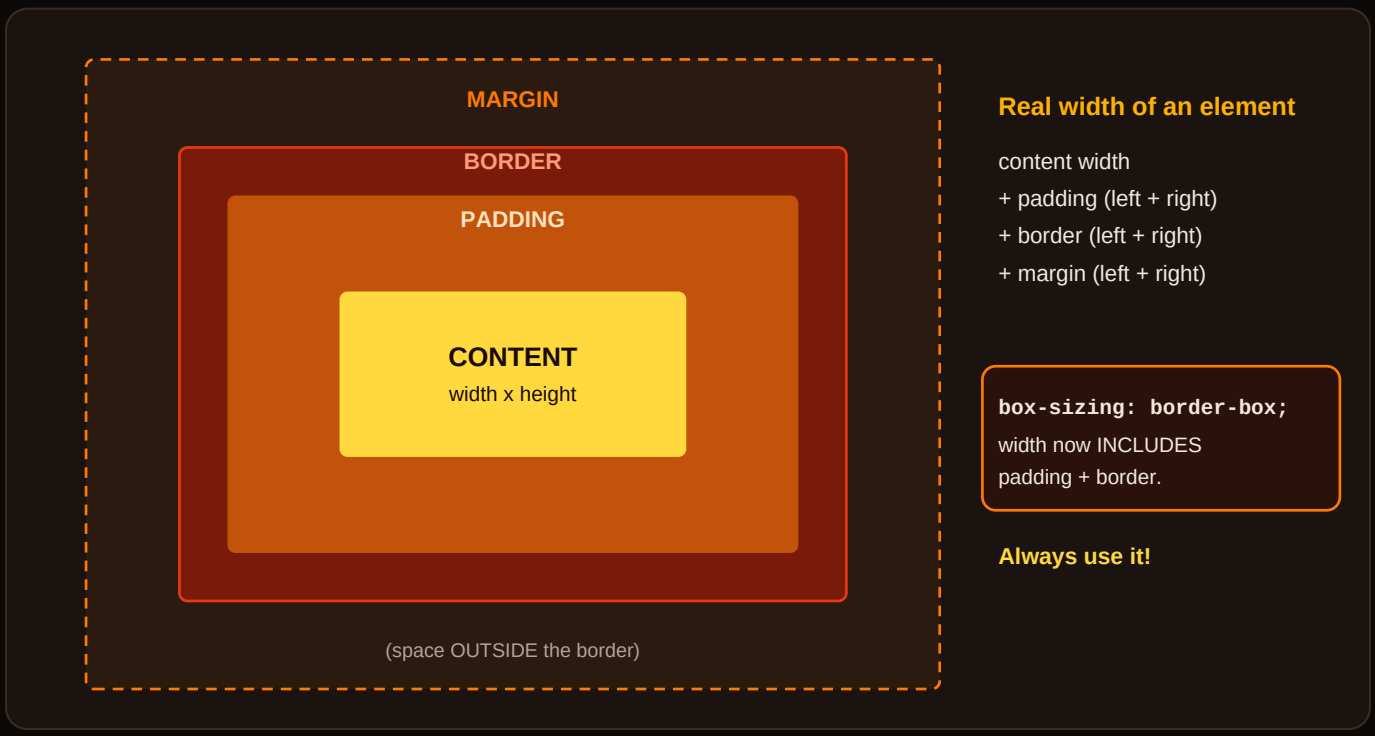


Figure 8 - Content, padding, border and margin

CSS

```css
*, *::before, *::after { box-sizing: border-box; }   /* put this in EVERY project */

.card {
  width: 300px;
  padding: 20px;                /* all four sides                   */
  border: 2px solid #ff7a00;    /* width style colour               */
  margin: 16px auto;            /* top/bottom 16px, left/right auto = centred */
}
/* shorthand order: top right bottom left (clockwise)
   margin: 10px 20px 30px 40px;   margin: 10px 20px;  (vertical horizontal) */
```

WATCH OUT **Margin collapsing:** vertical margins of neighbouring blocks merge into the larger one (20px + 30px = 30px, not 50px). Use **gap**, padding or flex/grid to avoid surprises.

Backgrounds, Borders and Shadows

CSS

```css
.hero {
  background-color: #0b0807;
  background-image: url("../images/hero.jpg");
  background-size: cover;            /* fill the box, crop if needed */
  background-position: center;
  background-repeat: no-repeat;
}
.card {
  border-radius: 14px;               /* 50% on a square = circle     */
  box-shadow: 0 8px 24px rgba(255, 122, 0, 0.35);   /* x y blur colour */
  outline: 2px dashed gold;          /* like border, takes no space  */
  overflow: hidden;                  /* clip anything that spills out */
  opacity: 0.9;
}
img { max-width: 100%; height: auto; object-fit: cover; }
```

Display and Visibility

Value	Effect
`display: block`	Full-width box on its own line. Width and height apply.
`display: inline`	Flows in text. Width, height and vertical margin are ignored.
`display: inline-block`	Flows in text but accepts width, height and margin.
`display: none`	Removed from the page entirely (takes no space).
`visibility: hidden`	Invisible but still takes up space.
`display: flex` / `grid`	Turns the element into a layout container for its children.

Positioning and z-index

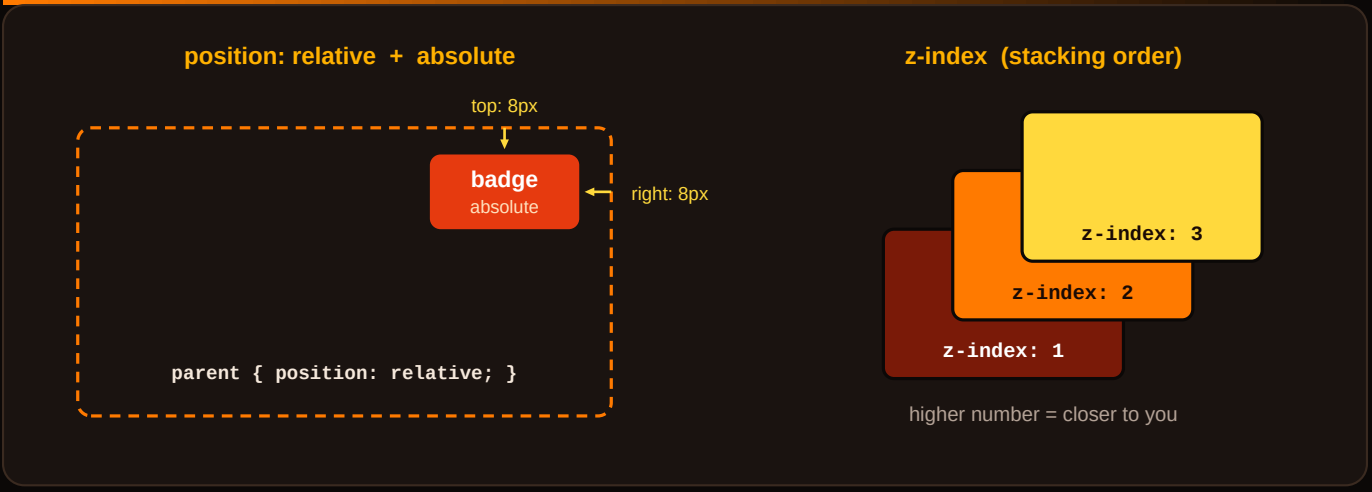


Figure 9 - A relatively positioned parent anchors an absolute child; z-index orders layers

position	Behaviour
`static`	Default. Normal flow. `top`, `left`, `z-index` are ignored.
`relative`	Normal flow, but can be nudged with `top` / `left`. **Becomes the anchor for absolute children.**
`absolute`	Removed from flow, placed relative to the nearest positioned ancestor.

position	Behaviour
`fixed`	Stuck to the viewport (sticky header, floating button). Ignores scrolling.
`sticky`	Scrolls normally, then sticks at an offset: `position: sticky; top: 0;`.

CSS

```css
.card { position: relative; }
.card .badge {
  position: absolute;
  top: 8px;
  right: 8px;
  z-index: 2;                /* higher = on top (needs a position value) */
}
header { position: sticky; top: 0; z-index: 100; }
```

Flexbox: One-Dimensional Layout

Flexbox lays children out in a **row or a column** and distributes space between them. It is the best tool for navbars, button groups, cards in a row and centring things.



Figure 10 - Flex axes and justify-content values

CSS

```css
.row {
  display: flex;
  flex-direction: row;              /* row (default) | column | row-reverse */
  justify-content: space-between;   /* spread along the MAIN axis           */
  align-items: center;              /* align along the CROSS axis           */
  flex-wrap: wrap;                  /* let items drop to the next line      */
  gap: 16px;                        /* space between items                  */
}
.row .grow { flex: 1; }             /* take up the remaining space          */

/* The famous perfect centre */
.center {
  display: flex; justify-content: center; align-items: center;
  min-height: 100vh;
}
```

Property	Goes on	Does
`display: flex`	container	Activates flexbox.
`flex-direction`	container	Sets the main axis: `row` or `column`.
`justify-content`	container	Distributes items along the main axis.
`align-items`	container	Aligns items along the cross axis (`stretch`, `center`, `flex-start`, `flex-end`).
`flex-wrap`, `gap`	container	Wrap onto new lines; spacing between items.
`flex: 1` / `flex-grow`	item	How much extra space the item takes.
`align-self`, `order`	item	Override alignment for one item; change visual order.

🔥 Grid: Two-Dimensional Layout

CSS Grid controls **rows and columns at the same time**. Use it for full page layouts and card galleries. Use Flexbox inside components, Grid for the page structure.

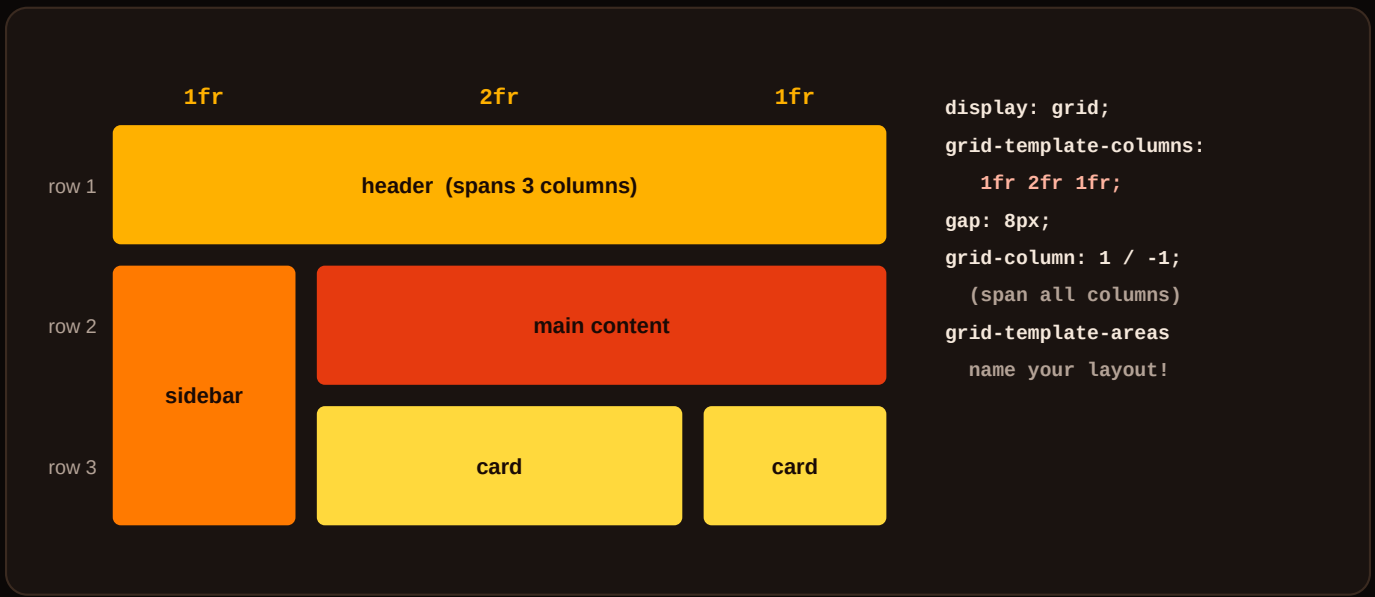


Figure 11 - A grid with named tracks and a spanning header

CSS

```css
.layout {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;    /* three columns          */
  grid-template-rows: auto 1fr auto;
  gap: 16px;
}
.layout .header { grid-column: 1 / -1; }  /* span from line 1 to the last line */

/* Name the areas: the layout is visible right in the CSS */
.page {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-areas:
    "header  header"
    "sidebar main"
    "footer  footer";
  min-height: 100vh;
}
.page > header { grid-area: header; }
.page > aside  { grid-area: sidebar; }
.page > main   { grid-area: main; }
.page > footer { grid-area: footer; }
```

PRO TIP Flexbox = content decides the layout (1D). Grid = you decide the layout first (2D). They work together: a grid page whose cards are flex containers.

Interactive CSS: Hover, Transitions, Transforms

You can add a lot of life with **zero JavaScript**. A transition animates a property change smoothly instead of snapping.

CSS

```css
.btn {
  background: #e63a0f;
  color: white;
  padding: 0.7rem 1.4rem;
  border-radius: 999px;
  transition: transform 0.25s ease, box-shadow 0.25s ease, background 0.25s;
}
.btn:hover {
  background: #ff7a00;
  transform: translateY(-3px) scale(1.05);       /* lift + grow          */
  box-shadow: 0 10px 28px rgba(255, 122, 0, 0.55);
}
.btn:active { transform: translateY(0) scale(0.98); }
.btn:focus-visible { outline: 3px solid #ffd93d; outline-offset: 3px; }
```

Transform	Effect
`translate(x, y)`	Move without affecting layout.
`scale(1.2)`	Grow / shrink.
`rotate(45deg)`	Rotate.
`skew(10deg)`	Slant.

PRO TIP Animate only `transform` and `opacity` for 60 fps smoothness. Animating `width`, `height` or `top` forces the browser to recalculate layout every frame.

Keyframe Animations: The Fire Flicker

CSS

```css
@keyframes flicker {
  0%, 100% { transform: scale(1) rotate(-1deg);    filter: brightness(1);   }
  25%      { transform: scale(1.06, 0.97) rotate(1deg);  filter: brightness(1.2); }
  50%      { transform: scale(0.97, 1.08) rotate(-2deg); filter: brightness(0.95); }
  75%      { transform: scale(1.04) rotate(1deg);    filter: brightness(1.15); }
}
.flame {
  animation: flicker 1.4s ease-in-out infinite;
  transform-origin: bottom center;
  text-shadow: 0 0 20px #ff7a00, 0 0 40px #e63a0f;
}
@media (prefers-reduced-motion: reduce) {
  .flame { animation: none; }       /* respect users who dislike motion */
}
```

animation property	Meaning
animation-name	The **@keyframes** to use.
animation-duration	How long one cycle lasts (**1.4s**).
animation-timing-function	**ease**, **linear**, **ease-in-out**, **cubic-bezier(...)**.
animation-iteration-count	Number of repeats or **infinite**.
animation-direction / **-delay** / **-fill-mode**	Alternate direction, wait before start, keep end state.

CSS Variables (Custom Properties)

CSS

```css
:root {
  --fire-red: #e63a0f;
  --fire-orange: #ff7a00;
  --bg: #0b0807;
  --text: #f2e8df;
  --space: 1rem;
  --radius: 12px;
}
.card {
  background: var(--bg);
  color: var(--text);
  border: 1px solid var(--fire-orange);
  padding: var(--space);
  border-radius: var(--radius);
}

/* Automatic light / dark mode */
@media (prefers-color-scheme: light) {
  :root { --bg: #fff8f1; --text: #1b110b; }
}
```

Change a variable once and the whole site updates. Variables are the foundation of themes and design systems.

Modern CSS Worth Knowing

CSS

```css
.hero-title { font-size: clamp(2rem, 5vw + 1rem, 4.5rem); }   /* min, ideal, max   */
.thumb      { aspect-ratio: 16 / 9; object-fit: cover; }      /* keep proportions  */
.container  { width: min(100% - 2rem, 1100px); margin-inline: auto; }
:is(h1, h2, h3):hover { color: var(--fire-orange); }          /* shorter grouping  */
.card:has(img) { padding: 0; }                                /* parent selector   */
html { scroll-behavior: smooth; }                              /* smooth anchors    */
```

A Solid Reset / Base Stylesheet

css/style.css (top of file)

```css
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
html { scroll-behavior: smooth; -webkit-text-size-adjust: 100%; }
body {
  font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
  line-height: 1.6;
  min-height: 100vh;
}
img, video, svg { display: block; max-width: 100%; height: auto; }
a { color: inherit; }
ul[class], ol[class] { list-style: none; }
button, input, select, textarea { font: inherit; color: inherit; }
```

Debugging CSS with DevTools

1. Right-click the element > **Inspect**. The Elements panel opens on it.
2. In **Styles**, see every rule that applies. Struck-through rules lost the cascade.
3. Untick a declaration or edit a value live. Changes are temporary, so copy the fix into your file.
4. Open the **Computed** tab for the box-model diagram and final values.
5. Hover an element in the DOM tree to highlight its margin / padding / content on the page.
6. Add `outline: 1px solid red;` to everything (`* { outline: 1px solid red; }`) to see all the boxes.

CSS Common Mistakes

Symptom	Likely cause and fix
My style does not apply.	Typo in the class name, wrong file path in `<link>`, or a more specific rule wins. Inspect it.
Element is wider than the screen.	Missing `box-sizing: border-box`, or a fixed `width` in px. Use `max-width: 100%`.
Cannot centre a div.	`margin: 0 auto` needs a `width`; or use `display: flex; justify-content: center`.
`z-index` does nothing.	The element has `position: static`. Add `position: relative`.
`height: 100%` does nothing.	The parent has no height. Use `min-height: 100vh` or a flex/grid parent.
Unwanted gap under an image.	Images are inline. Add `img { display: block; }`.
Hover works on desktop, not on phones.	Touch has no hover. Never hide essential info behind hover only.

PART 04

RESPONSIVE DESIGN

The full guideline: one codebase, every screen

What is Responsive Design?

People open your site on 5-inch phones, tablets, laptops and 32-inch monitors. **Responsive design** means one set of HTML and CSS that adapts itself to every screen, instead of separate sites for separate devices.

It stands on three ingredients:

- **Fluid layouts:** sizes in %, `fr`, `rem` and `max-width` instead of fixed pixel widths.
- **Flexible media:** images and videos that shrink with their container.
- **Media queries:** CSS rules that apply only above or below certain screen widths.

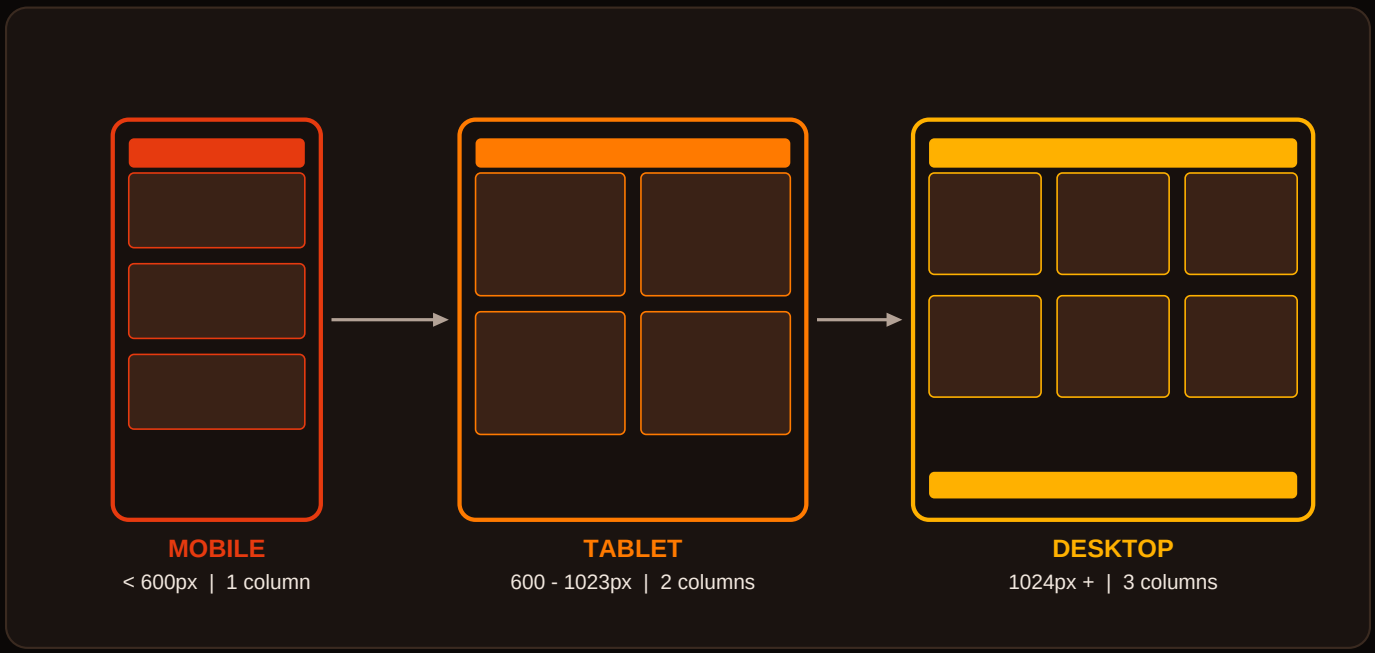


Figure 12 - The same content re-arranges itself as the screen grows

The 10-Step Responsive Workflow

1. Write **semantic HTML** first and put the content in a sensible reading order.
2. Add the **viewport meta tag** to the head.
3. Add the **reset** (`box-sizing: border-box`, fluid images).
4. Style for the **smallest screen first** (about 360px wide). One column, big tap targets.
5. Use `rem` for type and spacing, `%` / `fr` / `max-width` for widths.
6. Slowly widen the browser. **When the design breaks, add a breakpoint** (not by device name).
7. Use `@media (min-width: ...)` to add columns and bigger spacing.
8. Make images responsive: `max-width: 100%; height: auto;`.
9. Test in DevTools device mode **and on a real phone**.

10. Check accessibility: zoom to 200%, tab through with the keyboard, check contrast.

Step 1: The Viewport Meta Tag

```HTML
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Without it, phones pretend to be about 980px wide and shrink your desktop layout down to fit, so your media queries never trigger. With it, the layout width equals the real device width. **It is the single most important line for mobile.** Never add `user-scalable=no`, because it blocks pinch zoom and harms accessibility.

Mobile-First vs Desktop-First

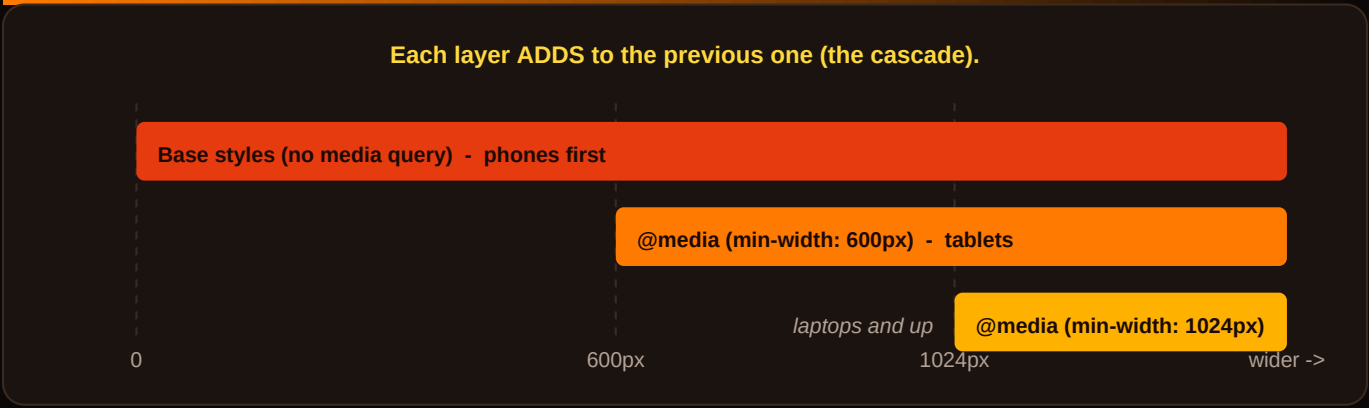


Figure 13 - Mobile-first: start simple, then layer on enhancements

	Mobile-first (recommended)	Desktop-first
Query style	`min-width` (grow up)	`max-width` (shrink down)
Base CSS	Simple single column for phones	Full desktop layout
Benefit	Less CSS, faster phones, forces focus on content	Easy to start when you have a desktop design
Risk	None really	Phones download CSS they override

```CSS
/* 1. BASE: phones. No media query needed. */
.cards { display: grid; gap: 1rem; grid-template-columns: 1fr; }

/* 2. TABLETS and up */
@media (min-width: 600px) {
  .cards { grid-template-columns: repeat(2, 1fr); }
}

/* 3. LAPTOPS and up */
@media (min-width: 1024px) {
  .cards { grid-template-columns: repeat(3, 1fr); gap: 1.5rem; }
}
```

Media Queries and Breakpoints

CSS

```css
@media (min-width: 768px) { ... }                    /* width >= 768px          */
@media (max-width: 767px) { ... }                    /* width <= 767px          */
@media (min-width: 600px) and (max-width: 1023px) { ... }   /* a range         */
@media (orientation: landscape) { ... }              /* wider than tall         */
@media (prefers-color-scheme: dark) { ... }          /* user uses dark mode     */
@media (prefers-reduced-motion: reduce) { ... }      /* user dislikes motion    */
@media (hover: hover) { .btn:hover { ... } }         /* device has a real mouse */
@media print { nav, footer { display: none; } }      /* when printing           */
```

Breakpoint	Targets	Typical change
Base (0+)	Phones, 320-599px	One column, stacked nav / burger menu, large tap targets.
`600px`	Large phones, small tablets	Two-column card grids, bigger padding.
`768px`	Tablets portrait	Horizontal navigation replaces the burger.
`1024px`	Laptops, tablets landscape	Three columns, sidebar layouts.
`1280px`+	Desktops	Cap the width with `max-width`, add more whitespace.

PRO TIP These numbers are starting points. Breakpoints should follow **your content**: widen the browser until the layout looks stretched or broken, then add a breakpoint there.

🔥 Fluid Typography and Spacing

CSS

```css
:root {
  --step-0: clamp(1rem,    0.95rem + 0.3vw, 1.125rem);   /* body    */
  --step-2: clamp(1.5rem,  1.2rem  + 1.5vw, 2.25rem);    /* h2      */
  --step-4: clamp(2.2rem,  1.5rem  + 3.5vw, 4.5rem);     /* h1      */
  --space: clamp(1rem, 4vw, 3rem);                       /* padding */
}
body { font-size: var(--step-0); }
h1   { font-size: var(--step-4); line-height: 1.1; }
section { padding-block: var(--space); }
```

`clamp(min, preferred, max)` scales smoothly with the screen but never goes below the minimum or above the maximum. No media query needed.

🔥 Responsive Images and Video

CSS

```css
/* CSS: the one-line fix for fluid images */
img, video { max-width: 100%; height: auto; display: block; }

/* Cropping to a fixed ratio without distortion */
.thumb { width: 100%; aspect-ratio: 4 / 3; object-fit: cover; }

/* Responsive 16:9 video embed */
.video { aspect-ratio: 16 / 9; }
.video iframe { width: 100%; height: 100%; border: 0; }
```

HTML

```
<!-- Let the browser pick the best file for the screen -->
<img src="images/hero-800.jpg"
     srcset="images/hero-400.jpg 400w, images/hero-800.jpg 800w,
             images/hero-1600.jpg 1600w"
     sizes="(min-width: 1024px) 50vw, 100vw"
     alt="Developer working at night" loading="lazy">

<!-- Art direction: different crop for phones -->
<picture>
  <source media="(min-width: 768px)" srcset="images/banner-wide.webp">
  <img src="images/banner-square.webp" alt="Fire banner">
</picture>
```

🔥 Layout Patterns That Always Work

Pattern 1: The centred container

CSS

```
.container {
  width: min(100% - 2rem, 1100px);   /* 1rem gutters on phones, 1100px cap */
  margin-inline: auto;               /* centred */
}
```

Pattern 2: Stack on mobile, row on desktop (Flexbox)

CSS

```
.split { display: flex; flex-direction: column; gap: 1.5rem; }
@media (min-width: 768px) {
  .split { flex-direction: row; align-items: center; }
  .split > * { flex: 1; }
}
```

Pattern 3: Auto-fit card grid (no media queries!)

CSS

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.25rem;
}
/* As many 240px+ columns as fit, they share the remaining space equally. */
```

Pattern 4: Sidebar layout

CSS

```
.with-sidebar { display: grid; gap: 1.5rem; }
@media (min-width: 900px) {
  .with-sidebar { grid-template-columns: 260px 1fr; }
}
```

Pattern 5: Sticky footer

CSS

```
body { min-height: 100vh; display: flex; flex-direction: column; }
main { flex: 1; }        /* main grows, footer stays at the bottom */
```

Responsive Navigation

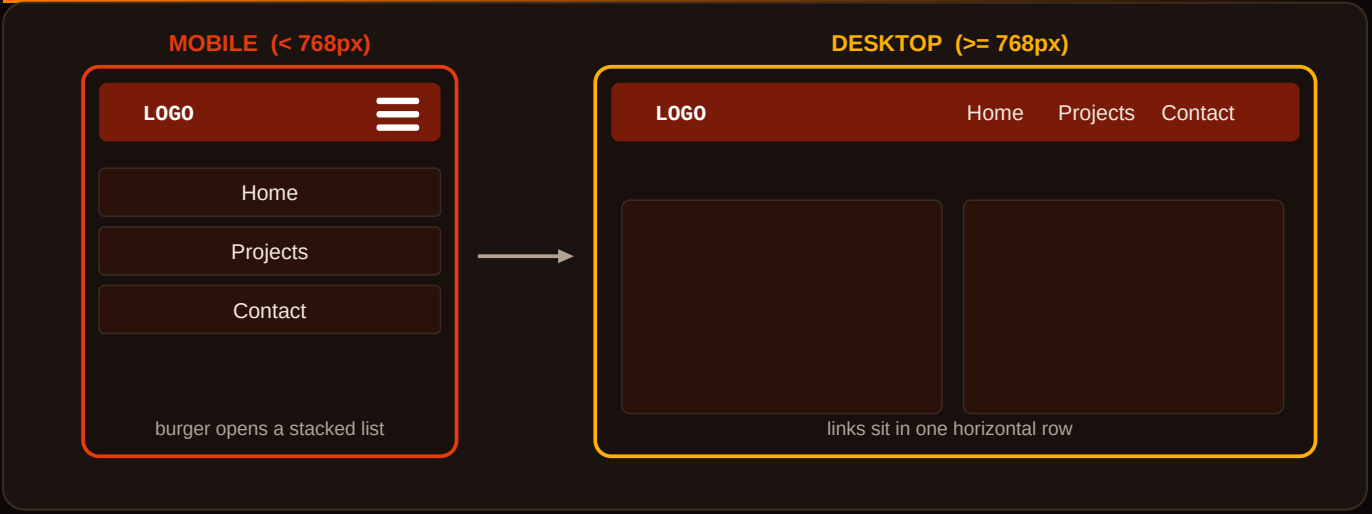


Figure 14 - Burger menu on phones, inline links on larger screens

HTML

```html
<header class="site-header">
  <a href="index.html" class="logo">Sheharyar</a>
  <input type="checkbox" id="menu-toggle" class="menu-toggle">
  <label for="menu-toggle" class="burger" aria-label="Toggle menu">&#9776;</label>
  <nav class="nav">
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="projects.html">Projects</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </nav>
</header>
```

CSS

```css
.site-header { display: flex; flex-wrap: wrap; align-items: center;
               justify-content: space-between; padding: 1rem; }
.menu-toggle { display: none; }                 /* hidden checkbox = the switch */
.burger { font-size: 1.8rem; cursor: pointer; }
.nav { display: none; width: 100%; }            /* menu closed on phones */
.menu-toggle:checked ~ .nav { display: block; } /* checkbox ticked = menu open */
.nav ul { list-style: none; display: flex; flex-direction: column; gap: 0.5rem; }

@media (min-width: 768px) {
  .burger { display: none; }
  .nav { display: block; width: auto; }
  .nav ul { flex-direction: row; gap: 1.5rem; }
}
```

PRO TIP This burger menu uses **zero JavaScript**. In the JS module you will rebuild it with a button and `classList.toggle()` for more control.

Tables, Forms and Touch Targets

CSS

```css
.table-wrap { overflow-x: auto; }          /* wide tables scroll inside their box */
table { border-collapse: collapse; min-width: 480px; width: 100%; }

input, select, textarea, button {
  width: 100%; font-size: 1rem;            /* 16px+ stops iOS zooming on focus */
  padding: 0.8rem 1rem; border-radius: 10px;
}
a, button { min-height: 44px; }            /* comfortable thumb-sized target */
```

- Tap targets should be at least **44 x 44px** with space between them.
- Keep body text at **16px or larger** on phones.
- Keep line length near **45-75 characters**: `max-width: 65ch`.
- Wrap data tables in a scrolling container instead of shrinking them to unreadable size.

A Complete Mobile-First Page

index.html (body)

```html
<body>
  <header class="site-header"> ... nav ... </header>
  <main>
    <section class="hero container">
      <h1>I build <span class="fire">fast</span> things for the web.</h1>
      <p>AI and full-stack engineer in the making.</p>
      <a class="btn" href="projects.html">See my work</a>
    </section>
    <section class="container">
      <h2>Skills</h2>
      <div class="cards">
        <article class="card"><h3>HTML</h3><p>Semantic structure.</p></article>
        <article class="card"><h3>CSS</h3><p>Responsive layouts.</p></article>
        <article class="card"><h3>JS</h3><p>Coming soon.</p></article>
      </div>
    </section>
  </main>
  <footer class="site-footer container"><p>&copy; 2026 Sheharyar</p></footer>
</body>
```

css/style.css (complete starter)

```css
:root { --red: #e63a0f; --orange: #ff7a00; --bg: #0b0807;
        --card: #1a1310; --text: #f2e8df; }
* { box-sizing: border-box; margin: 0; }
body { background: var(--bg); color: var(--text); font-family: system-ui, sans-serif;
       line-height: 1.6; min-height: 100vh; display: flex; flex-direction: column; }
main { flex: 1; }
.container { width: min(100% - 2rem, 1100px); margin-inline: auto; }
.hero { padding-block: clamp(3rem, 10vw, 7rem); }
.hero h1 { font-size: clamp(2.2rem, 1.5rem + 3.5vw, 4.5rem); line-height: 1.1; }
.fire { background: linear-gradient(to top, var(--red), #ffd93d);
        -webkit-background-clip: text; background-clip: text; color: transparent; }
.btn { display: inline-block; margin-top: 1rem; padding: 0.8rem 1.6rem;
       background: var(--orange); color: #1a0a05; font-weight: 700;
       border-radius: 999px; text-decoration: none; transition: transform 0.2s; }
.btn:hover { transform: translateY(-3px); }
.cards { display: grid; gap: 1rem;
         grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }
.card { background: var(--card); border: 1px solid #3a2a20;
        border-radius: 14px; padding: 1.2rem; }
.card:hover { border-color: var(--orange); box-shadow: 0 8px 24px rgba(255,122,0,.3); }
```

Responsive Testing Checklist

- [ ] Viewport meta tag is present in every page.
- [ ] Tested at **320px**, **375px**, **768px**, **1024px** and **1440px** in DevTools.
- [ ] No horizontal scrollbar at any width (check for fixed px widths).
- [ ] Images never overflow their container and never look stretched.
- [ ] Text is readable without zooming; line length is comfortable.
- [ ] All buttons and links are easy to tap (44px) and not too close together.
- [ ] Navigation works on touch AND with the keyboard.
- [ ] Tested on one real phone (not only the simulator).
- [ ] Page still works when zoomed to 200% and with larger system fonts.
- [ ] Landscape orientation checked.

Responsive Mistakes to Avoid

Mistake	Fix
Fixed `width: 1200px` on containers.	Use `max-width: 1200px; width: 100%`.
Breakpoints named after devices ("iPhone 14").	Base them on where the layout breaks.
`max-width` desktop-first queries that override half the CSS.	Write mobile-first with `min-width`.
Tiny text and tiny tap targets on phones.	16px body text, 44px targets.
Hiding important content on mobile with `display: none`.	Rearrange it; do not delete it.
Huge unoptimised images.	Resize, use WebP, add `srcset` and `loading="lazy"`.
Forgetting `box-sizing: border-box`.	Add the universal reset first.

PART 05

PROJECT ARCHITECTURE

Structure your portfolio like a professional

Project Folder Structure

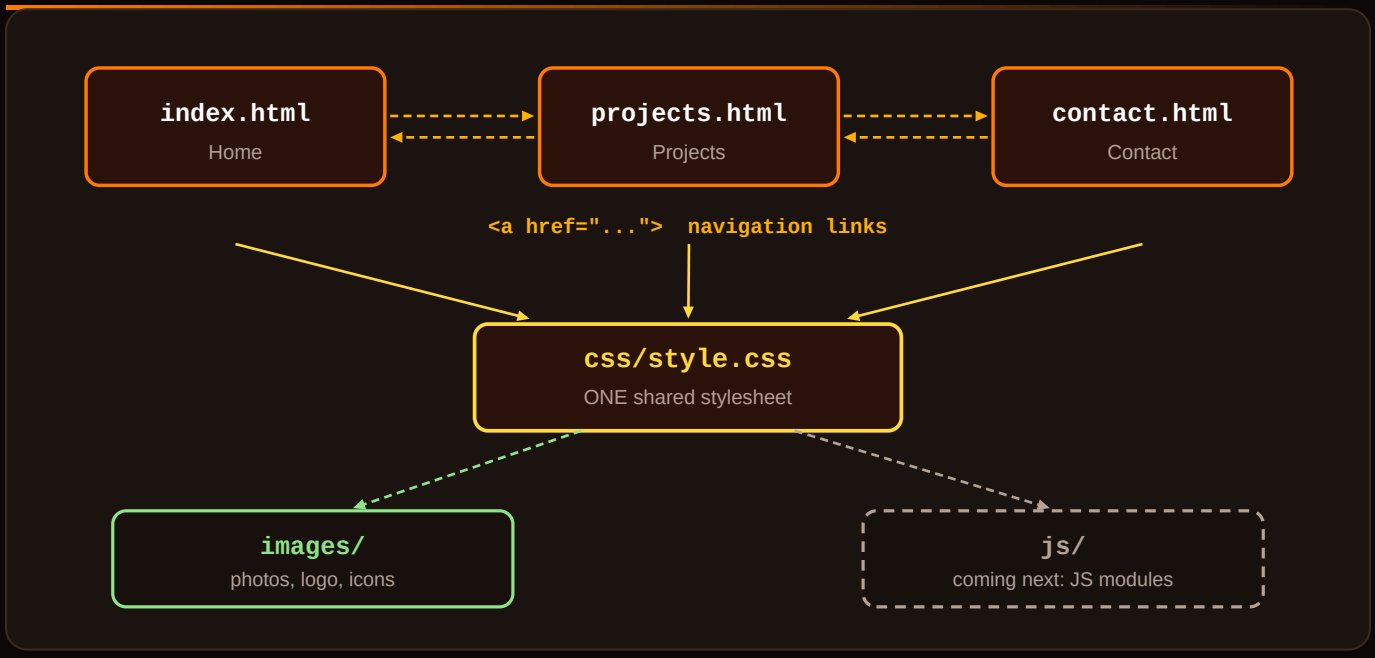


Figure 15 - Three pages share one stylesheet and one image folder

Recommended structure

```
portfolio/
|-- index.html              <- Home page (must be named index.html)
|-- projects.html           <- Projects page
|-- contact.html            <- Contact page
|-- README.md               <- What the project is + how to run it
|-- .gitignore
|-- css/
|   `-- style.css           <- ONE shared stylesheet
|-- images/
|   |-- favicon.png
|   |-- profile.webp
|   `-- project-1.webp
`-- js/                     <- empty for now (next module)
    `-- app.js
```

- One responsibility per folder: pages in the root, styles in `css/`, media in `images/`, scripts in `js/`.
- Lowercase names, hyphens instead of spaces, descriptive (`project-weather-app.webp`).
- Use **relative paths** (`css/style.css`), never absolute paths from your own computer (`C:/Users/...`).

Page Plan: What Goes on Each Page



Figure 16 - Wireframes: plan the layout on paper before you write code

Page	Sections	Key skills practised
index.html	Header + nav, hero (photo, name, tagline, call to action), skills cards, short about, footer	Flexbox, gradients, hover effects, typography
projects.html	Title, responsive grid of project cards (image, title, description, link), footer	CSS Grid `auto-fit`, images, cards, transitions
contact.html	Title, contact form (name, email, message, button), social links, footer	Forms, labels, validation attributes, focus states

The Shared HTML Skeleton

Every page repeats the same header and footer. Copy this skeleton, then change only the `<title>`, the `<main>` content and which link is marked as the current page.

projects.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Projects | Sheharyar Sarmad</title>
  <meta name="description" content="Projects built by Sheharyar Sarmad.">
  <link rel="icon" href="images/favicon.png">
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="site-header container">
    <a class="logo" href="index.html">SS</a>
    <nav>
      <ul class="nav-list">
        <li><a href="index.html">Home</a></li>
        <li><a href="projects.html" aria-current="page">Projects</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main class="container"> <!-- page-specific content --> </main>

  <footer class="site-footer container"><p>&copy; 2026 Sheharyar Sarmad</p></footer>
</body>
</html>
```

CSS Architecture: One File, Clear Order

CSS

```
/* ==== 1. Variables ============ :root { --colours, --spacing, --radius } */
/* ==== 2. Reset ================ box-sizing, margins, fluid images        */
/* ==== 3. Base ================= body, headings, links, paragraphs        */
/* ==== 4. Layout =============== .container, .site-header, .site-footer  */
/* ==== 5. Components =========== .btn, .card, .nav-list, .form           */
/* ==== 6. Page sections ======== .hero, .skills, .projects, .contact     */
/* ==== 7. Utilities ============ .text-center, .mt-2, .visually-hidden   */
/* ==== 8. Media queries ======== mobile-first, smallest to largest       */
```

Naming	Example	Why
BEM: Block__Element--Modifier	`.card`, `.card__title`, `.card--featured`	Self-documenting, low specificity, no clashes.
Class by purpose, not looks	`.btn-primary` not `.big-red-button`	Survives redesigns.
Hyphen-case	`.site-header`	Standard CSS convention.

Reusable Components

CSS

```
/* a reusable card */
.card { background: var(--card); border: 1px solid #3a2a20; border-radius: var(--radius);
        overflow: hidden; transition: transform .25s, box-shadow .25s; }
.card__img { width: 100%; aspect-ratio: 16 / 9; object-fit: cover; }
.card__body { padding: 1rem 1.2rem 1.4rem; }
.card__title { font-size: 1.15rem; margin-bottom: .4rem; }
.card:hover { transform: translateY(-6px); box-shadow: 0 14px 30px rgba(255,122,0,.3); }
.card--featured { border-color: var(--orange); }
```

HTML

```
<article class="card card--featured">
  <img class="card__img" src="images/project-1.webp" alt="Weather app screenshot">
  <div class="card__body">
    <h3 class="card__title">Weather App</h3>
    <p>Responsive weather dashboard built with HTML and CSS.</p>
    <a class="btn" href="https://github.com/Sheharyar-Sarmad">View code</a>
  </div>
</article>
```

Git Workflow for This Project

Terminal

```
git init
git add .
git commit -m "feat: add project structure and index.html"
git commit -m "style: responsive hero section and navigation"
git commit -m "feat: projects page with card grid"
git commit -m "fix: contact form focus styles"
git branch -M main
git remote add origin https://github.com/YOUR-NAME/portfolio.git
git push -u origin main
```

Commit small and often. Prefixes like `feat`, `fix`, `style`, `docs` and `refactor` make your history easy to read, which recruiters notice.

Deploy It for Free

Service	How	Result
GitHub Pages	Repo > Settings > Pages > Deploy from branch `main` / root	`your-name.github.io/portfolio`
Netlify	Drag and drop the folder, or connect your repo	Free `.netlify.app` URL + HTTPS
Vercel	Import the GitHub repo	Free `.vercel.app` URL, automatic redeploys

Put the live link in your README, on your GitHub profile and on LinkedIn.

Final Quality Checklist

- [ ] All pages validate with the W3C validator (validator.w3.org).
- [ ] Every image has `alt` text; every input has a `label`.
- [ ] One `h1` per page with a logical heading order.
- [ ] Navigation links work on every page and the current page is marked.
- [ ] No console errors and no 404s in the Network tab.
- [ ] Responsive from 320px to 1440px with no horizontal scroll.
- [ ] Lighthouse scores above **90** for Performance, Accessibility, Best Practices and SEO.
- [ ] README explains the project, shows a screenshot and includes the live link.
- [ ] Deployed and tested from a different device.

PART 06

EXERCISES

Practice is where the learning actually happens

How to Use This Section

- Work through the levels **in order**. Tick each checkbox (they work in this PDF) when you finish.
- Type every line yourself. **No copy-paste, no AI-generated code** for the exercises.
- When you are stuck: read the error, inspect with DevTools, check the docs, then ask.
- Commit after every exercise: `git commit -m "exercise 1.3 done"`.
- Create one folder per exercise (`ex-01-recipe`, `ex-02-bio-card` ...).

Level 0: Web Basics (answer in your notebook)

- [ ] Explain in your own words what happens between typing a URL and seeing the page (5 steps).
- [ ] What is the difference between the internet and the web?
- [ ] What does DNS do? What would happen if it was down?
- [ ] Name 3 HTTP methods and 3 status codes with their meaning.
- [ ] Frontend vs backend: list two jobs of each.
- [ ] What are the roles of HTML, CSS and JavaScript? Use an analogy of your own.
- [ ] Open any site, press F12 and find: the page title in the DOM, one CSS rule, one request in the Network tab.
- [ ] Install VS Code + Live Server + Prettier and run a page on a local server.

Level 1: HTML Exercises

- [ ] **Bio page:** `h1`, two paragraphs, an image with `alt`, a list of 5 hobbies, and a link to your GitHub that opens in a new tab safely.
- [ ] **Recipe page:** title, photo in a `figure` with `figcaption`, ingredients in a `ul`, steps in an `ol`, and a `blockquote` tip.
- [ ] **Timetable:** a `table` with `caption`, `thead`, `tbody`, `th scope`, and one cell using `colspan`.
- [ ] **Contact form:** name, email, password, date, radio group, checkbox, `select`, `textarea`, a submit button. Every input needs a label and at least three need validation attributes.
- [ ] **Semantic blog layout:** `header`, `nav`, `main` with two `article`s, `aside`, `footer`. No `div` allowed.
- [ ] **Multi-page:** three linked pages with relative links and an in-page "back to top" anchor.
- [ ] **Media page:** embed a YouTube video with `iframe`, a local `video` with `controls`, and an `audio` player.
- [ ] **Validate:** run every page through validator.w3.org and fix all errors and warnings.

Level 2: CSS Exercises

- [ ] Restyle the bio page with an external stylesheet: fonts from Google Fonts, a colour palette in CSS variables, comfortable `line-height` and `max-width: 65ch`.
- [ ] Build a **profile card**: image with `border-radius: 50%`, name, role, button, `box-shadow`, and a hover lift with `transition`.
- [ ] Draw the **box model** for one element on paper, then change `margin`, `padding` and `border` in DevTools and watch it update.
- [ ] Create a **navbar** with Flexbox: logo left, links right, hover underline animation, `position: sticky`.
- [ ] Create a **3-card row** with Flexbox that wraps to a column on narrow screens (`flex-wrap`).
- [ ] Build the **holy grail layout** (header, sidebar, main, footer) using `grid-template-areas`.
- [ ] Recreate a **pricing section** with three plan cards; the middle one is larger and highlighted.
- [ ] Write a **fire button**: gradient background, glowing `box-shadow` on hover, and a `@keyframes` pulse animation.
- [ ] Create a **dark / light theme** by overriding variables inside `@media (prefers-color-scheme: light)`.
- [ ] Break your own page on purpose (wrong selector, missing `position`), then find and fix each bug using only DevTools.

Level 3: Responsive Design Exercises

- [ ] Take the bio page and make it perfect at **360px, 768px and 1280px**, mobile-first.
- [ ] Build a **responsive card gallery** with `repeat(auto-fit, minmax(240px, 1fr))` and no media queries.
- [ ] Build the **burger navigation** from this PDF, then restyle it your own way.
- [ ] Make a hero heading and spacing scale smoothly with `clamp()`.
- [ ] Add `srcset` + `sizes` to an image and confirm in the Network tab that phones download a smaller file.
- [ ] Make a wide table scroll horizontally inside its own container on phones.
- [ ] Put a **sidebar layout** on desktop that stacks below the content on mobile.
- [ ] Run Lighthouse on your page and raise Accessibility and Performance above 90.

Mini Projects

Project	What you build	Skills	Level
Tribute page	A one-page tribute to a person you admire	Semantic HTML, images, typography	Easy
Product card	A shop card with image, price, button, hover effects	Box model, flexbox, transitions	Easy
Landing page	Hero, features, testimonials, call-to-action, footer for a made-up product	Grid, flexbox, responsive, gradients	Medium
Pricing page	Three plans with a toggled highlight	Grid, hover, `:has`, variables	Medium
Photo gallery	Responsive masonry-style gallery with hover captions	Grid, `object-fit`, positioning	Medium
Clone a site section	Recreate a section of a real site pixel-by-pixel	Everything	Hard

Frontend Mentor Practice Path

1. Create a free account at frontendmentor.io and open **Challenges**.
2. Filter by **Newbie** or **Junior**, and by HTML + CSS only.
3. Download the starter files (design images, assets, style guide). Do not look at the solutions yet.
4. Build mobile layout first, then desktop, measuring with the supplied design.
5. Push the finished project to GitHub, deploy it and submit the live URL for feedback.

6. Complete at least **3 challenges** before the portfolio, then move up a difficulty level.

Final Challenge: The Fire Portfolio

> **REMEMBER** Build a **2-3 page personal portfolio** completely by hand. No templates, no AI-written code. Look things up in docs, but type every line. This is the proof you can build static websites.

- [ ] Pages: `index.html`, `projects.html`, `contact.html` (an extra `about.html` is a bonus).
- [ ] Folder structure exactly as shown in Part 05, one shared `css/style.css`.
- [ ] Semantic HTML with a shared header, nav and footer on every page.
- [ ] CSS variables for colours, spacing and radius; a consistent theme of your choice.
- [ ] Mobile-first and responsive from 320px to 1440px; burger or stacked nav on phones.
- [ ] At least: one Flexbox layout, one Grid layout, hover + focus states, and one transition or animation.
- [ ] A working-looking contact form with labels and validation attributes.
- [ ] All images optimised with `alt`, `width`, `height` and `loading="lazy"`.
- [ ] Pushed to GitHub with at least 10 meaningful commits and a good README.
- [ ] Deployed on GitHub Pages / Netlify and the link added to your LinkedIn and README.

Rubric	Points	What earns full marks
HTML structure and semantics	20	Valid, semantic, accessible, logical headings.
CSS quality and organisation	20	Variables, reset, clear sections, low specificity, no inline styles.
Responsive design	20	Mobile-first, no overflow, fluid type and images.
Layout skills	15	Confident use of Flexbox and Grid in the right places.
Visual design and polish	15	Consistent theme, spacing, hover states, a touch of animation.
Git, README and deployment	10	Clean commits, screenshots, live URL.
Total	**100**	**85+ = ready for JavaScript. Below 70 = redo the weak area first.**

Self-Test Quiz

Try to answer each question **before** you look at the answers on the next page.

1. What is the difference between a `class` and an `id`?
2. Which tag must appear exactly once per page: `h1` or `main`? (Hint: both!)
3. What does `box-sizing: border-box` change?
4. Order these by specificity: `p`, `.note`, `#intro`, inline style.
5. What does `justify-content` control in Flexbox? And `align-items`?
6. What is the difference between `display: none` and `visibility: hidden`?
7. Why must an absolutely positioned element have a positioned parent?
8. What does the viewport meta tag do and why is it essential?
9. Mobile-first: do you use `min-width` or `max-width` queries?
10. What does `repeat(auto-fit, minmax(240px, 1fr))` do?
11. Which two CSS properties are best for smooth 60 fps animations?
12. Why should every image have an `alt` attribute?

Quiz Answers

#	Answer
1	`id` is unique (one element per page); `class` is reusable on many elements and is what you style with.
2	Both: one `h1` for the page topic and one visible `main` for the main content.
3	Width and height include padding and border, so boxes stop growing past the size you set.
4	Inline style (1000) > `#intro` (100) > `.note` (10) > `p` (1).
5	`justify-content` aligns along the main axis; `align-items` along the cross axis.
6	`display: none` removes the element and its space; `visibility: hidden` hides it but keeps the space.
7	Absolute positions are calculated from the nearest positioned ancestor; otherwise it uses the page.
8	It makes the layout width equal the device width, so phones do not zoom out and media queries trigger.
9	`min-width`: write the phone styles as the base and add enhancements for larger screens.
10	Creates as many columns as fit (min 240px each) and shares leftover space equally. Responsive with no media query.
11	`transform` and `opacity`.
12	Screen readers read it aloud, it shows if the image fails to load, and search engines use it.

One-Page Cheat Sheet

Goal	Snippet
Reset sizing	`*, *::before, *::after { box-sizing: border-box; }`
Fluid image	`img { max-width: 100%; height: auto; display: block; }`
Centre a block	`.box { width: min(100% - 2rem, 1100px); margin-inline: auto; }`
Perfect centre	`display: flex; justify-content: center; align-items: center;`
Row that wraps	`display: flex; flex-wrap: wrap; gap: 1rem;`
Auto card grid	`grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));`
Span full row	`grid-column: 1 / -1;`
Full-screen section	`min-height: 100vh;`
Sticky header	`position: sticky; top: 0; z-index: 100;`
Fluid font	`font-size: clamp(1.5rem, 1rem + 3vw, 3rem);`
Mobile-first query	`@media (min-width: 768px) { ... }`
Smooth hover	`transition: transform .25s ease; } :hover { transform: translateY(-4px); }`
Gradient text	`background: linear-gradient(...); background-clip: text; color: transparent;`
Truncate text	`white-space: nowrap; overflow: hidden; text-overflow: ellipsis;`
Visually hidden	`position: absolute; width: 1px; height: 1px; overflow: hidden; clip-path: inset(50%);`

What Comes Next: JavaScript

Your pages look great but they are **static**. Next you make them react: menus that open, forms that validate, content that loads from APIs. The JavaScript track is split into two levels.

Module	Level	You will learn
js_basics	Beginner	Variables, types, operators, conditions, loops, functions, arrays, objects, the DOM, events, forms.
js_advance	Advanced	Scope and closures, `this`, prototypes and classes, ES6+, modules, promises, `async/await`, fetch API, error handling.

After that: **TypeScript, Node.js and Express, MERN / PERN, Next.js**, then full-stack projects and AI-powered apps. The portfolio you build now becomes the home for all of them.

> **PRO TIP** Finished the portfolio? Add it to your GitHub README and LinkedIn **Featured** section, then start `js_basics`. Keep the fire burning.